esprit

Se former autrement

2022 - 2023

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : INFORMATION TECHNOLOGY

TITLE

By: *(full name)*

Academic supervisor: *(full name)*

Corporate Internship Supervisor: *(full name)*

LOGO ENTREPRISE

cdio

Délivrée par: Cti

EUR-ACE

CONFÉRENCE DES GRANDES ÉCOLES

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : ……..

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : ……..

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : ……..

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

Contents

1 General Introduction **1**
- 1.1 General Context . . . 1
- 1.2 Host Organization . . . 1
  - 1.2.1 Presentation of Coral-io . . . 1
  - 1.2.2 Internship Framework . . . 2
- 1.3 Problem Statement . . . 2
  - 1.3.1 Limitations of Existing Approaches . . . 2
  - 1.3.2 The Core Gap . . . 3
  - 1.3.3 Formal Problem Statement . . . 3
- 1.4 Proposed Solution . . . 3
- 1.5 Project Objectives and Scope . . . 4
  - 1.5.1 Objectives . . . 4
  - 1.5.2 Scope . . . 5
- 1.6 Project Methodology and Work Planning . . . 5
  - 1.6.1 Development Phases . . . 5
  - 1.6.2 Rationale for the Approach . . . 6

2 State of the Art **7**
- 2.1 Introduction . . . 7
- 2.2 Algorithmic Trading: Evolution and Challenges . . . 7
- 2.3 Market Regime Detection . . . 7
  - 2.3.1 Rule-Based Approaches . . . 7
  - 2.3.2 Statistical Approaches: Hidden Markov Models and Clustering . . . 8
  - 2.3.3 Deep Learning Approaches . . . 8
- 2.4 LLM Agents in Finance . . . 8
  - 2.4.1 Single- and Multi-Agent LLM Trading Systems . . . 8
  - 2.4.2 Multi-Agent Debate as a Design Pattern . . . 9
  - 2.4.3 Limitations of Existing Approaches . . . 9
- 2.5 Reinforcement Learning for Trading . . . 9
  - 2.5.1 Early Approaches: DQN and Discrete Action Spaces . . . 9
  - 2.5.2 PPO and Continuous Action Spaces . . . 9
  - 2.5.3 Curriculum Learning in Finance . . . 10
- 2.6 Positioning of MARTA . . . 10
- 2.7 Summary . . . 10

3 Methodology **12**
- 3.1 Overview . . . 12
- 3.2 Data and Feature Engineering . . . 12

3.2.1 Data Sources and Coverage . . . 12
3.2.2 Pipeline Stages . . . 13
3.2.3 Train / Validation / Test Split . . . 13
3.3 Regime Encoder . . . 14
3.3.1 Design Objective . . . 14
3.3.2 Architecture . . . 15
3.3.3 Contrastive Pair Construction . . . 16
3.3.4 Loss Function . . . 16
3.3.5 Training Procedure . . . 16
3.3.6 Ongoing Refinement: Pooling and Pair Construction . . . 17
3.4 Regime Detection . . . 17
3.4.1 Design Objective . . . 17
3.4.2 Dimensionality Reduction . . . 17
3.4.3 Model Selection . . . 17
3.4.4 Final Fit and Regime Stream Generation . . . 18
3.5 Multi-Agent Deliberation System . . . 18
3.5.1 Design Objective . . . 18
3.5.2 Orchestration and LLM Backend . . . 19
3.5.3 Regime Context . . . 19
3.5.4 Agent Roster . . . 19
3.5.5 Debate Flow . . . 19
3.5.6 Risk Limit Computation . . . 20
3.5.7 Fund Manager Synthesis . . . 20
3.6 RL Execution Agent . . . 20
3.6.1 Design Objective . . . 20
3.6.2 MDP Formulation . . . 21
3.6.3 Policy Network . . . 21
3.6.4 Curriculum Training . . . 22
3.6.5 Evaluation Determinism . . . 22
3.7 Evaluation Methodology . . . 22
3.7.1 Metrics . . . 22
3.7.2 Ablation Design . . . 23
3.7.3 Statistical Testing . . . 23
3.7.4 In-Sample Regime Behavior Diagnostic . . . 23
3.8 Dashboard and Live System . . . 23
3.8.1 Design Objective . . . 23
3.8.2 Architecture . . . 23
3.8.3 Live Regime Lookup . . . 24
3.8.4 Live Feature Bridge . . . 24
3.8.5 Auditability . . . 24

Chapter 1

General Introduction

1.1 General Context

Financial markets process trillions of dollars in transactions daily across equities, currencies, commodities, and digital assets. Over the past two decades, algorithmic trading has evolved from a niche practice used by quantitative hedge funds into the dominant force in global markets, now accounting for the majority of daily trading volume on major exchanges.

The core appeal is clear: algorithms process data at speeds no human can match and are immune to emotional bias. Yet building a consistently profitable system remains one of the hardest problems in applied machine learning. Markets are non-stationary — their statistical properties shift over time. A strategy that excels during a trending bull market may fail entirely in a volatile, mean-reverting environment. This sensitivity to changing market conditions is not a minor detail; it is the central challenge of the field.

Early rule-based systems lacked adaptability. Machine learning, and reinforcement learning in particular, offered a path forward by framing trading as a sequential decision problem. More recently, large language models introduced the ability to reason over unstructured data — news, macroeconomic commentary, sentiment — opening a new dimension in trading research. However, existing systems that combine these approaches share a critical blind spot: none condition their decisions on a *learned* representation of the current market regime. This gap defines the motivation for MARTA.

1.2 Host Organization

1.2.1 Presentation of Coral-io

Coral-io is a technology consulting firm specializing in digital transformation and IT solutions across multiple industries. The company provides comprehensive services that bridge technical expertise with business objectives, helping clients navigate complex technological challenges. Its activity spans five main areas:

- **DevOps Services**: CI/CD pipeline implementation, Infrastructure as Code, containerization and orchestration (Docker, Kubernetes), and test automation.
- **Application Performance Management (APM)**: full-stack observability, real-time monitoring, and bottleneck diagnosis for production software.

- **Third-Party Solution Integration**: as a Micro Focus (OpenText) partner, Coral-io delivers IT Operations Management (ITOM) integration and legacy system modernization.
- **Research & Development**: proof-of-concept development and emerging-technology adoption, sitting at the intersection of technical expertise and applied innovation.
- **Finance and Insurance Consulting**: FinTech implementation, regulatory compliance, and digital transformation for financial services clients.

MARTA was carried out within the R&D and Finance/Insurance Consulting scope of the company's activity, combining applied AI research with a financial-markets use case.

1.2.2 Internship Framework

Unlike a project built around a pre-defined internal need, this final-year project was conducted under a self-directed research framework: Coral-io granted full autonomy in selecting the subject and technical direction of the work, provided it aligned with the company's R&D and FinTech interests. This arrangement reflects a model already familiar from the author's prior internship experience at Coral-io, where independent, exploratory project work was similarly supported as a way to build internal expertise in emerging domains.

Within this framework, technical guidance was provided collaboratively by Ms. Oumaima Aounallah and the wider Coral-io team, consulted on an as-needed basis for architectural decisions, tooling questions, and periodic progress reviews rather than through prescriptive task assignment. Academic supervision was provided by Saoussen Lakhdhar at ESPRIT.

This structure placed a greater share of responsibility for scoping, planning, and technical decision-making on the author than is typical of a company-assigned project — a distinction elaborated further in Section 1.6.

1.3 Problem Statement

1.3.1 Limitations of Existing Approaches

Three broad families of automated trading systems exist today, each with a distinct blind spot relevant to MARTA.

Rule-based and classical statistical strategies (moving-average crossovers, mean-reversion bands, momentum filters) are transparent and computationally cheap, but their parameters are fixed at design time. When the market regime shifts — from trending to ranging, from low to high volatility — these fixed parameters no longer match the underlying data-generating process, and performance degrades until the strategy is manually re-tuned.

Pure reinforcement learning agents address this rigidity by learning a policy directly from historical price action, allowing some adaptation to changing dynamics. However, standard RL formulations treat the market as a single, undifferentiated environment: the agent is not explicitly told which regime it is currently operating in, and must infer this implicitly through its state representation. In practice, this means the

policy tends to average its behavior across regimes rather than specializing for each one, and offers no human-interpretable explanation for why a given action was chosen.

LLM-based trading agents, a more recent development, exploit large language models' ability to reason over unstructured text — news headlines, macroeconomic reports, analyst commentary — to produce qualitative market assessments. This adds a reasoning layer absent from purely numerical approaches, and can in principle justify its decisions in natural language. However, existing systems in this category typically condition the LLM's reasoning only on recent price statistics or raw text, without any structured, learned signal describing the broader statistical regime the market is currently in.

1.3.2 The Core Gap

Across all three families, no system combines (i) an explicit, learned representation of market regime, (ii) multi-perspective reasoning informed by that representation, and (iii) an execution layer that adapts its risk behavior according to regime and reasoning confidence. Regime is either ignored entirely, treated as a hidden variable an RL agent must discover implicitly, or left to ad hoc heuristics (e.g., a simple volatility threshold) rather than learned from data.

1.3.3 Formal Problem Statement

Given a multi-asset time series of market data, the problem addressed by this work is twofold:

1. **Regime representation learning**: learn a low-dimensional, temporally-aware embedding of market state that captures latent structure predictive of near-term dynamics, without relying on hand-crafted regime labels.

2. **Regime-conditioned decision making**: use this learned representation to condition both a multi-perspective reasoning process and a downstream execution policy, such that trading behavior adapts to the identified regime rather than applying a single fixed strategy across all market conditions.

Success is measured not only by trading performance (risk-adjusted returns, drawdown) but by whether regime conditioning demonstrably improves outcomes relative to regime-agnostic baselines — the central empirical question this report answers in Chapter **??**.

1.4 Proposed Solution

MARTA (Multi-Agent Regime-Aware Trading Architecture) addresses this gap through a five-layer pipeline, each layer directly targeting one part of the problem statement above.

1. **Regime Encoder**: a Temporal Fusion Transformer (TFT) trained with an InfoNCE contrastive objective learns dense embeddings of market state from hourly multi-asset data (BTC, ETH, EUR/USD, XAU/USD), without requiring hand-labeled regimes.

2. **Regime Detection**: a Gaussian Hidden Markov Model is fitted over the encoder's embedding space, discretizing the continuous representation into a small number of interpretable, statistically distinct regimes with associated transition dynamics.

3. **Multi-Agent Deliberation**: five specialized LLM agents debate the appropriate trading stance given the current regime, each contributing a different analytical perspective (e.g., trend, risk, contrarian), producing a reasoned, regime-aware recommendation rather than a black-box signal.

4. **RL Execution Supervisor**: a PPO-trained agent translates the agents' deliberation output into concrete position sizing and execution decisions, learning to modulate risk exposure according to regime and the confidence of the upstream reasoning.

5. **Monitoring Dashboard**: a Next.js interface exposes regime detection, agent reasoning, and execution decisions in real time, making the system's behavior auditable rather than opaque.

This layered design directly mirrors the two-part problem statement: layers 1–2 solve the regime representation learning problem, and layers 3–4 solve the regime-conditioned decision making problem. Layer 5 is a supporting component providing interpretability rather than a decision-making stage in its own right. The empirical contribution of regime-awareness across layers 1–4 is evaluated through dedicated ablation studies, presented in Chapter **??**.

1.5 Project Objectives and Scope

1.5.1 Objectives

The project pursues four concrete objectives, each corresponding to a layer of the proposed solution:

1. **Learn a meaningful regime representation**: train a TFT + InfoNCE encoder on multi-asset hourly data such that the resulting embedding space captures structure relevant to near-term market dynamics, validated through downstream regime separability rather than reconstruction loss alone.

2. **Discover interpretable, statistically distinct regimes**: fit a Gaussian HMM over the learned embeddings and verify that the resulting regimes are stable, distinguishable, and align with intuitive market conditions (e.g., trending vs. ranging, high vs. low volatility) upon inspection.

3. **Build a regime-aware multi-agent reasoning layer**: design and implement a five-agent LLM deliberation system whose recommendations are conditioned on detected regime, and evaluate whether this conditioning improves recommendation quality over a regime-agnostic baseline.

4. **Train an adaptive execution policy**: implement a PPO-based execution supervisor that translates agent deliberation into position-sizing decisions, and evaluate — via ablation — whether regime-awareness measurably improves risk-adjusted trading performance relative to an equivalent regime-agnostic system.

A secondary, supporting objective is to expose the full pipeline through a monitoring dashboard, so that regime detection, agent reasoning, and execution decisions remain auditable rather than opaque — a property largely absent from prior work in this space (Section 1.3.1).

1.5.2 Scope

In scope:

- Four assets: BTC/USD, ETH/USD, EUR/USD, XAU/USD, at hourly resolution, 2018–2025.
- Design, training, and evaluation of all five architectural layers described in Section 1.4.
- Backtesting and ablation studies quantifying the contribution of regime-awareness specifically.
- A functional monitoring dashboard for the full pipeline.

Out of scope:

- Live deployment with real capital; the system is evaluated via backtesting and paper-trading simulation only.
- Assets or timeframes beyond the four instruments and hourly resolution listed above.
- Regulatory, compliance, or brokerage-integration work required for production trading.
- Optimization of LLM inference cost/latency for high-frequency use cases; the multi-agent layer operates at hourly decision frequency, not tick-level.

This scope reflects the project's nature as an applied research contribution demonstrating the value of regime-awareness, rather than a production-ready trading product.

1.6 Project Methodology and Work Planning

Given the exploratory, research-oriented nature of MARTA, a rigid upfront specification was not appropriate: each architectural layer's design depended on empirical results from the layer built before it (e.g., the number and structure of latent regimes discovered by the HMM directly shaped how agent prompts were engineered). The project therefore followed an **iterative, layer-by-layer methodology**, where each phase produced a working, evaluable component before the next was started, rather than a big-bang integration at the end.

1.6.1 Development Phases

The work was organized into seven successive phases, summarized in Table 1.1 and Figure 1.1. The first phase, covering literature review, problem definition, and architecture design, produced Chapter 2's survey and the five-layer design introduced in Section 1.4; the remaining six phases correspond one-to-one with the architectural layers and validation work described in Chapter 3.

Table 1.1: MARTA development phases

Phase	Deliverable	Duration & dates
1. Literature Review & Architecture Design	State-of-the-art survey, 5-layer design	12 wk (Mar 1 – May 24)
2. Data & Feature Engineering	Cleaned multi-asset hourly dataset (2018–2025)	2 wk (May 25 – Jun 7)
3. Regime Encoder	Trained TFT + InfoNCE encoder	3 wk (Jun 8 – Jun 28)
4. Regime Detection	Gaussian HMM over encoder embeddings	1 wk (Jun 29 – Jul 5)
5. Multi-Agent System	5-agent LLM debate pipeline	2 wk (Jul 6 – Jul 19)
6. RL Execution Agent	PPO supervisor trained on agent outputs	3 wk (Jul 20 – Aug 9)
7. Dashboard & Validation	Next.js dashboard, live system, ablations	3 wk (Aug 10 – Aug 31)

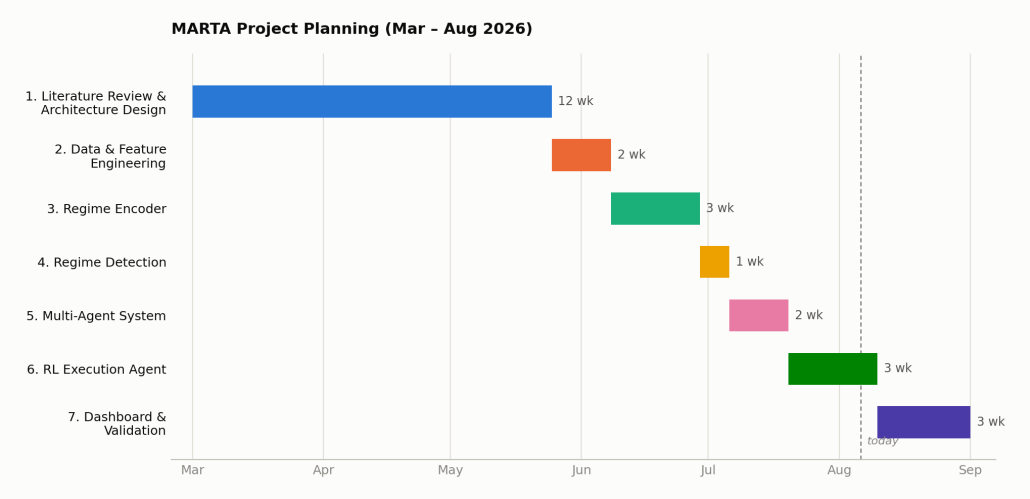


Figure 1.1: Project planning Gantt chart

1.6.2 Rationale for the Approach

This phase-by-phase structure offered two advantages over a monolithic design-then-build approach. First, it allowed early validation of the encoder and regime detection stages in isolation — verifying, for instance, that discovered regimes were interpretable and stable before committing to an agent architecture built on top of them. Second, it kept technical risk contained: had the HMM failed to produce meaningful regime clusters, the multi-agent and RL layers could have been redesigned without discarding the data pipeline or encoder work.

The sections of Chapter 3 follow this same phase ordering, each one corresponding to a development stage described above.

Chapter 2

State of the Art

2.1 Introduction

This chapter surveys the three bodies of work MARTA draws on and positions itself against: market regime detection, LLM-based trading agents, and reinforcement learning for trading. Section 2.2 briefly traces how algorithmic trading arrived at its current state. Sections 2.3–2.5 review each of the three families in turn, closing each with the specific limitation that motivates one part of MARTA's design. Section 2.6 then consolidates these gaps into a single comparative view and states precisely where MARTA sits relative to prior work.

2.2 Algorithmic Trading: Evolution and Challenges

Automated trading systems have progressed through three broad generations. The first generation consisted of rule-based strategies — moving-average crossovers, momentum filters, mean-reversion bands — valued for their transparency but structurally unable to adapt once market conditions diverge from the assumptions baked into their fixed parameters. The second generation applied supervised and reinforcement learning to trading directly, allowing strategies to be learned from data rather than hand-specified, at the cost of interpretability. The third and most recent generation incorporates large language models, exploiting their capacity to reason over unstructured text such as news and analyst commentary rather than numerical features alone. Each generation solved a limitation of the one before it while introducing a new one, a pattern examined in detail across the following three sections.

2.3 Market Regime Detection

2.3.1 Rule-Based Approaches

The simplest regime-detection methods rely on hand-crafted thresholds over indicators such as realized volatility or moving-average slope to classify the market into discrete states (e.g., trending vs. ranging). These approaches are computationally trivial and easy to interpret, but the thresholds themselves must be manually tuned per asset and require periodic re-calibration as market conditions evolve, making them brittle outside the conditions they were tuned on.

2.3.2 Statistical Approaches: Hidden Markov Models and Clustering

Hidden Markov Models (HMMs) have long served as the primary statistical tool for regime detection in finance, following the foundational regime-switching framework introduced by Hamilton for modeling business-cycle turning points in macroeconomic time series [1]. Applied to financial returns, an HMM treats the market regime as an unobserved latent state that generates observable return and volatility patterns, with transition probabilities between states estimated directly from data rather than imposed by rule. Gaussian HMMs fitted over hand-engineered feature sets (returns, realized volatility) have been shown to recover interpretable bull/bear/consolidation regimes on major equity indices, and have been used as risk-managing filters that suppress trading during predicted high-volatility states. K-means and Gaussian Mixture clustering offer a simpler, non-sequential alternative, but typically yield only two well-separated clusters (bullish/bearish) and lack the transition-probability structure that makes HMMs useful for characterizing regime persistence and switching dynamics.

The central limitation of this family, for MARTA's purposes, is the feature representation the HMM operates on: virtually all published work fits the HMM directly over hand-selected statistical features (returns, volatility, sometimes volume), rather than over a representation learned to capture broader temporal structure. This caps the ceiling of what the HMM can discover — it can only find regimes expressible in terms of the features it was given.

2.3.3 Deep Learning Approaches

More recent work combines regime detection with deep reinforcement learning rather than treating it as a standalone preprocessing step. A 2025 study integrating an HMM-based regime filter directly into the FinRL reinforcement learning framework found that regime-aware hybrid agents outperformed standard regime-agnostic FinRL baselines on risk-adjusted return metrics across a 30-stock portfolio, providing direct empirical support for the core hypothesis underlying MARTA: that explicit regime conditioning improves trading outcomes relative to regime-agnostic learning. However, this and comparable work still fits the HMM over hand-engineered statistical features rather than a learned temporal representation, and pairs regime detection only with an RL execution layer — it does not incorporate any reasoning layer capable of explaining *why* a regime implies a particular stance, which is precisely the gap MARTA's multi-agent layer is designed to fill.

2.4 LLM Agents in Finance

2.4.1 Single- and Multi-Agent LLM Trading Systems

TradingGPT, an early multi-agent system, assigns LLM agents layered memory and distinct behavioral characters to simulate diverse trader personas contributing to a shared decision. FinAgent, presented at KDD 2024, is a multimodal foundation agent that fuses textual news, numerical price data, and visual chart information through a dual-level reflection module, allowing the agent to adapt its strategy based on recent market feedback; evaluated across six financial datasets spanning equities and crypto, it sub-

stantially outperformed prior single-modality baselines. TradingAgents, proposed shortly after, more directly mirrors the organizational structure of a real trading firm: specialized analyst agents (fundamental, sentiment, technical) feed a research team of Bull and Bear debaters, whose output passes through a risk management team before a trader agent commits to a decision, and the resulting framework outperformed several classical baselines (Buy-and-Hold, MACD, RSI) on cumulative return and Sharpe ratio in simulated equity trading.

2.4.2 Multi-Agent Debate as a Design Pattern

The common thread across TradingGPT, FinAgent, and TradingAgents is the use of multiple specialized agents whose outputs are synthesized rather than a single monolithic LLM call — a design pattern motivated by the observation that decomposing analysis into distinct perspectives (technical, sentiment, risk) produces more robust recommendations than asking one model to reason about all of them simultaneously. MARTA's five-agent deliberation layer adopts this same debate pattern.

2.4.3 Limitations of Existing Approaches

None of the systems reviewed above condition their agents' reasoning on a learned, structured representation of market regime. TradingAgents' agents reason over recent price statistics and text directly; FinAgent's reflection module adapts based on recent performance feedback rather than an explicit regime signal; TradingGPT's characters are fixed personas rather than regime-conditioned roles. In each case, if the notion of "market regime" appears at all, it is implicit in whatever the LLM infers from raw inputs at inference time, not an explicit, pretrained signal shared across agents. This is the specific gap MARTA's regime encoder and HMM layer are designed to fill upstream of the agent debate.

2.5 Reinforcement Learning for Trading

2.5.1 Early Approaches: DQN and Discrete Action Spaces

Early applications of deep RL to trading adapted Deep Q-Networks (DQN) to discrete action spaces (buy/sell/hold), following the broader deep RL literature. FinRL, an open-source framework from Columbia University and NYU Shanghai, standardized this line of work by providing a full pipeline — market environments, fine-tuned implementations of DQN, DDPG, PPO, SAC, A2C, and TD3, and common reward functions — specifically to lower the engineering barrier for applying deep RL to quantitative trading. DQN-based agents are conceptually simple but constrained to discrete position sizes, limiting their ability to express graduated risk exposure.

2.5.2 PPO and Continuous Action Spaces

Proximal Policy Optimization (PPO), introduced by Schulman et al., addresses the instability of earlier policy gradient methods through a clipped surrogate objective that constrains the size of each policy update, achieving much of the stability of Trust Region Policy Optimization while being substantially simpler to implement. Its compatibility

with continuous action spaces makes it well suited to position-sizing tasks, where the agent must output a continuous fraction of capital to allocate rather than a discrete action, and it is the algorithm implemented for MARTA's execution supervisor for exactly this reason.

2.5.3 Curriculum Learning in Finance

Curriculum learning — structuring training so an agent first masters a simpler version of a task before progressing to a harder one — is comparatively underused in financial RL relative to other RL application domains, though the general principle (start with a single, liquid, well-behaved asset before extending to a more volatile or lower-liquidity one) aligns with standard practice in curriculum design more broadly. MARTA's own BTC-to-BTC+ETH curriculum training, detailed in Chapter 3, follows this logic.

2.6 Positioning of MARTA

Table 2.1 consolidates the gaps identified across the three families reviewed above.

Table 2.1: Comparison of MARTA against representative prior systems

System	**Learned regime repr.**	**Multi-agent reasoning**	**Adaptive executi**
HMM regime filters [2]	Partial (hand-features)	No	No
FinRL [3]	No	No	Yes (RL)
HMM + FinRL [4]	Partial (hand-features)	No	Yes (RL)
TradingGPT	No	Yes	No
FinAgent [5]	No	Partial (single agent)	No
TradingAgents [6]	No	Yes	No
MARTA (this work)	**Yes (TFT + InfoNCE)**	**Yes (5 agents)**	**Yes (PPO)**

No system reviewed combines a learned regime representation, multi-agent reasoning conditioned on that representation, and an adaptive execution layer in a single pipeline. This is precisely the combination MARTA proposes: a TFT encoder trained with an InfoNCE contrastive objective [7, 8] learns the regime representation that HMM-based approaches instead derive from hand-selected features; this representation conditions the multi-agent debate that LLM trading systems otherwise perform without any explicit regime signal; and a PPO-based execution supervisor [9] translates the resulting recommendation into position sizing, closing the loop that regime-detection-only systems leave open.

2.7 Summary

This chapter reviewed market regime detection, LLM-based trading agents, and reinforcement learning for trading as three largely separate research threads, each addressing part of the problem MARTA targets while leaving a specific gap: regime detection methods rely on hand-crafted features rather than learned representations; LLM agent systems reason without any explicit regime conditioning; and RL execution methods, even when regime-aware, lack an interpretable reasoning layer. Chapter 3 now turns to how MARTA's

five-layer architecture, introduced in Section 1.4, is designed and implemented to close these gaps jointly.

Chapter 3

Methodology

3.1 Overview

This chapter details the design and implementation of MARTA's five-layer architecture, introduced in Section 1.4 and developed across Phases 2–7 of the project plan (Table 1.1). Each section below corresponds to one development phase, in the same order the system was built: Section 3.2 covers the data pipeline (Phase 2), Section 3.3 the contrastive regime encoder (Phase 3), Section 3.4 the Gaussian HMM regime detector (Phase 4), Section 3.5 the five-agent LLM deliberation system (Phase 5), Section 3.6 the PPO execution agent (Phase 6), and Section 3.8 the monitoring dashboard and live-trading bridge (Phase 7). Section 3.7 describes the evaluation protocol — walk-forward splitting, the ablation design, and the statistical testing procedure — used to produce the results reported in Chapter **??**.

As established in Section 1.6, each layer's design was constrained by empirical results from the layer built before it; where a design decision was revised after an earlier choice proved inadequate, both the original approach and the reason for revising it are reported, since the revision itself is part of the project's methodology.

3.2 Data and Feature Engineering

3.2.1 Data Sources and Coverage

MARTA operates on four hourly instruments spanning two asset classes: **BTC/USDT** and **ETH/USDT**, fetched via the CCXT library against Binance, and **EUR/USD** and **XAU/USD** (gold), fetched from histdata.com's genuine 1-minute tick archives and resampled to hourly OHLCV (`open`=first, `high`=max, `low`=min, `close`=last, `volume`=sum). A fifth instrument, WTI crude oil, was dropped from the pipeline entirely: its exchange-holiday and weekend gaps were the binding constraint on cross-asset timestamp alignment, and removing it substantially improved the overlap between the crypto (24/7) and traditional-asset (weekday-only) series without any loss of coverage on the four remaining assets.

The crypto series are pulled forward continuously and reach mid-2026 at the time of writing; the forex/commodity series are capped at 2025-12-31 because histdata.com only publishes complete-year archives and no 2026 archive yet exists. This asymmetry is immaterial to the encoder, HMM, and RL training splits below (all of which end well before 2025), but is directly relevant to the live-trading bridge described in Section 3.8.

An earlier version of the EUR/USD and XAU/USD pipeline used daily yfinance data forward-filled to hourly resolution, which produced 23 near-duplicate hourly bars for every genuine daily observation and measurably contaminated the contrastive embedding space. Switching to histdata.com's genuine 1-minute data, resampled properly, removed this artifact.

3.2.2 Pipeline Stages

The pipeline runs as five sequential stages (`fetch` → `features` → `normalize` → `align` → `validate`), each operating on one asset's data independently until the alignment stage joins them:

1. **Fetch**: retrieves raw OHLCV via CCXT (crypto, batched request loop) or the histdata year-ZIP archive (forex/commodities), caching raw and intermediate artifacts to disk so repeated runs only fetch the missing tail. A sanitization pass enforces `high` = max(open,high,close) and `low` = min(open,low,close), and forward-fills zero-valued closes, guarding against malformed exchange data reaching any downstream stage.

2. **Features**: computes 13 technical indicators from the 5 raw OHLCV columns using the `ta` library and hand-rolled statistics, producing an 18-column intermediate frame (Table 3.1).

3. **Normalize**: applies a rolling 720-hour (30-day) z-score to 12 of the 13 indicators (minimum 168 hours of history required before a window is emitted), drops the raw OHLCV columns, and retains `close_return` unmodified as the return target. One indicator, `realized_vol_raw`, is deliberately excluded from normalization (see below).

4. **Align**: inner-joins all four assets' normalized frames on the BTC timestamp index (BTC/USDT trades 24/7 and is therefore the backbone), producing one combined parquet with MultiIndex columns `(asset, feature)` alongside per-asset parquets.

5. **Validate**: runs structural checks (column presence, dtype, no unexpected NaNs — CRITICAL severity), value-range checks, gap and distribution checks (WARNING severity), OHLCV sanity checks, and cross-asset timestamp-overlap checks, logging every metric to MLflow and writing a JSON validation report.

FX/commodity data from histdata.com carries no centralized volume figure (unlike a single-exchange crypto feed), so `obv_raw` and `vol_zscore` are near-constant for EUR/USD and XAU/USD; for these two assets, the price-derived indicators carry the useful signal.

3.2.3 Train / Validation / Test Split

All downstream components — encoder pretraining, HMM fitting, and RL training — share a single, strictly walk-forward split, defined once in a canonical module and imported everywhere it is needed rather than re-specified per component:

Split	**Window**
Train	2018-01-01 $\leq$ t $<$ 2024-01-01
Validation	2024-01-01 $\leq$ t $<$ 2025-01-01
Test	2025-01-01 $\leq$ t $<$ 2026-01-01

Table 3.1: The 14 features per asset used throughout the pipeline

Feature	Description
atr, adx, rsi, bb_width, macd_histogram	Standard technical indicators (Average True Range, Average Directional Index, Relative Strength Index, Bollinger Band width, MACD histogram), rolling z-scored.
obv_raw	On-Balance Volume, raw cumulative sum at feature-computation time; z-scored once (over 720h) at the normalize stage — an earlier version applied a second, redundant 168h z-score on top, which was removed.
vol_zscore, realized_vol, vol_ratio	Volatility-family indicators, rolling z-scored.
return_skew, return_kurtosis, autocorr_lag1	Higher-moment and autocorrelation statistics of returns, rolling z-scored.
realized_vol_raw	Unnormalized annualized realized volatility — the one feature the normalize stage does *not* z-score, so it preserves absolute magnitude. Included specifically because the z-scored realized_vol compresses extreme events (e.g. the March 2020 crash) into an ordinary-looking z-score, making them invisible to any downstream component that only sees the normalized value.
close_return	Simple hourly return, passed through unnormalized as the return target used by GARCH fitting and reward computation.

The three splits are contiguous by construction (the train/validation boundary equals the validation/test boundary of the previous pair, with no gap and no overlap), and this invariant is asserted at import time rather than merely documented. The test split is treated as sacred: no code path involved in encoder training, HMM/PCA fitting, or RL training or model selection is permitted to read a 2025 timestamp, and each of those three components independently re-verifies, after applying its own slice of the data, that the resulting slice does not cross the test boundary — catching a masking bug in the slicing logic itself, not just a mis-specified date constant. Test-set evaluation is performed exclusively through one dedicated, separate evaluation script, so that a mistake in an exploratory or diagnostic script can never silently contaminate model selection.

This three-way split, and the discipline around it, exists because every earlier iteration of the project used a shorter or differently-placed boundary (see Section 3.7); consolidating it into one canonical, self-validating module was a direct response to boundaries silently going stale in individual files during the several data-range extensions the project underwent.

3.3 Regime Encoder

3.3.1 Design Objective

Layer 1 of MARTA learns a fixed-length embedding of market state directly from the 14-feature, 60-candle window ending at each hourly timestamp, without relying on hand-labeled regimes. The embedding is trained with a self-supervised contrastive objective (InfoNCE [7]) so that two different *views* of the same underlying market window are

pulled together in embedding space while views of unrelated windows are pushed apart — the standard SimCLR-style contrastive pretraining recipe, applied here to multivariate financial time series instead of images.

3.3.2 Architecture

The encoder is *TFT-inspired* rather than a full Temporal Fusion Transformer [8]: it implements the core computational components of a TFT's encoder path — a Variable Selection Network (VSN), a recurrent sequence encoder, and a gated residual connection — but omits TFT's multi-horizon decoder, static covariate encoders, and interpretable multi-head attention block, none of which are needed for a fixed-window embedding task with no forecasting horizon. Table 3.2 lists the model's layers in forward-pass order.

Table 3.2: Regime encoder architecture (`TFTRegimeEncoder`)

Stage	**Shape**	**Description**
Input	$(B, 60, 14)$	60-hour window × 14 normalized features
VSN gate	$(B, 60, 14)$	Linear(14→14) + softmax over the feature axis at every timestep, elementwise-multiplied with the input — a learned, per-timestep feature-importance weighting.
LSTM encoder	$(B, 60, 128)$	2-layer `nn.LSTM`, hidden size 128, inter-layer dropout 0.1
Mean pool	$(B, 128)$	Unweighted mean over the 60 timesteps — chosen for robustness to regime length over, e.g., last-timestep pooling.
Gated residual	$(B, 128)$	$h \odot \sigma(\text{Linear}(h))$ — a sigmoid gate applied to the pooled representation, mirroring TFT's gated residual connections. This 128-dim vector is the embedding used everywhere downstream (Sections 3.4–3.6).
Projection head	$(B, 64)$	Linear(128→128) + ReLU + Linear(128→64) — used only by the contrastive loss during pretraining and discarded at inference time, following standard contrastive-learning practice (the projection head is known to distort the representation in ways that help the contrastive objective but do not help downstream tasks, so only the pre-projection embedding is kept).

The VSN's per-timestep softmax weights are also exposed directly (`get_vsn_weights`) and reused as the attention map consumed by the Technical Agent in Section 3.5, rather than re-deriving feature importance through a separate mechanism.

3.3.3 Contrastive Pair Construction

For each 60-candle window, a positive pair is formed by drawing two independently augmented views of the *same* window; all other windows in the mini-batch serve as negatives, matching the standard in-batch-negatives InfoNCE setup. Four stochastic augmentations are available, each applied independently with its own probability:

- **Amplitude scale** ($p = 0.5$): multiply the entire window by a scalar drawn from Uniform$(0.85, 1.15)$.
- **Jitter** ($p = 0.5$): add i.i.d. Gaussian noise, $\sigma = 0.03$, to every element.
- **Channel mask** ($p = 0.5$): zero out 2 randomly chosen feature columns for the whole window.
- **Time warp** ($p = 0.3$): resample the time axis along a smooth sinusoidal warp (random phase, amplitude up to 5 steps), implemented via per-feature linear interpolation onto the original 60-step grid; the warp function is analytically monotone for the chosen amplitude bound, so no timestep ordering is ever inverted.

3.3.4 Loss Function

The training objective is NT-Xent (the normalized temperature-scaled cross-entropy form of InfoNCE): both views' embeddings are L2-normalized, their pairwise cosine similarity matrix is scaled by a temperature τ, and cross-entropy is computed against the identity matching (each anchor's positive is its corresponding row/column index). Temperature is not fixed but learned, parameterized as $\tau = \exp(-\ell)$ with ℓ a trainable scalar initialized so that $\tau_0 = 0.07$; this lets gradient descent anneal the sharpness of the similarity distribution over training rather than committing to a hand-tuned constant.

An unconstrained learnable temperature can drift arbitrarily low, which collapses the contrastive loss into an near-trivial hard-argmax objective and was observed to happen during one training run (Section 3.7). The loss module therefore hard-clamps ℓ after every optimizer step so that τ cannot leave a configured $[\tau_{\min}, \tau_{\max}] = [0.03, 0.30]$ band, structurally rather than just via a post-hoc warning.

3.3.5 Training Procedure

Table 3.3: Encoder training hyperparameters

Parameter	**Value**	**Parameter**	**Value**
Window size	60 h	Batch size	512
Learning rate	1×10^{-3}	Weight decay	1×10^{-4}
Max epochs	200	Early-stopping patience	20 epochs
Warm-up	10 epochs	Gradient clip (max norm)	1.0

Training runs only over the train split defined in Section 3.2.3; validation loss on the 2024 window drives early stopping. Three convergence gates are checked at the end of training and recorded directly inside the saved checkpoint file (not only in the external experiment tracker, so that a checkpoint trained on a remote machine and copied back locally remains self-describing): validation loss below 4.0, top-1 contrastive retrieval accuracy above 0.20 (chance level is $1/512 \approx 0.002$ at this batch size), and learned temperature inside $[0.03, 0.30]$.

3.3.6 Ongoing Refinement: Pooling and Pair Construction

A later diagnostic pass compared the encoder's embeddings against simple raw-feature baselines on short-horizon return predictability and found the learned embedding underperforming, pointing to two candidate causes rather than outright representational collapse: mean-pooling across the full 60-step window may discard recency-relevant information that the LSTM's own per-timestep hidden states already contain, and augmenting two views of the *same* window (rather than pairing a window with a genuinely different, temporally adjacent one) may teach the encoder an invariance that is not useful for predicting near-term dynamics. Two isolated single-variable experiments are in progress to test these independently: swapping mean-pooling for last-timestep pooling on the already-trained checkpoint (no retraining required, since both reductions read from the same LSTM hidden states), and a CPC-style [7] variant that pairs each context window with its immediately following, non-overlapping future window instead of an augmented copy of itself. Neither experiment had produced a conclusive result at the time of writing; both are reported here as methodology under active revision rather than as a settled design choice.

3.4 Regime Detection

3.4.1 Design Objective

Layer 2 discretizes the continuous 128-dim embedding space from Layer 1 into a small number of interpretable regimes with explicit transition dynamics, using a Gaussian Hidden Markov Model [1] rather than a non-sequential clustering method, specifically to obtain the persistence and switching structure that downstream position-sizing and agent-conditioning logic depend on.

3.4.2 Dimensionality Reduction

Before HMM fitting, the 128-dim embedding is reduced to 16 dimensions via PCA, fit exclusively on training-split embeddings to avoid any leakage of validation or test structure into the projection. The pipeline asserts that the retained components explain more than 75% of training-set variance before proceeding; the production fit retains approximately 85%.

3.4.3 Model Selection

A GaussianHMM is fit independently for each candidate state count $n \in \{2, \ldots, 8\}$ using diagonal covariance, 100 EM iterations, tolerance 10^{-3}, on a fixed 20,000-row subsample

of the training embeddings (both fitting and scoring use the same subsample, so log-likelihoods are directly comparable across candidates). Each candidate is scored by BIC,

$$\mathrm{BIC}(n) = -2\,\mathrm{LL}(n) + \left(n^2 + 2n \cdot d\right) \ln(n_{\text{train}}),$$

where $d = 16$ is the PCA dimensionality and the parameter count n^2+2nd accounts for the transition matrix (n^2 entries) plus per-state diagonal mean and variance ($2d$ parameters per state). The candidate with the lowest BIC is not selected outright: a flat-curve guard instead selects the *smallest* n within 5% of the globally best BIC, on the reasoning that if several state counts fit the data almost equally well, the simplest one should be preferred. This guard was tightened during the project after an earlier version — which only compared the top two BIC candidates rather than every candidate against the global best — was found to produce a state count with duplicate, statistically indistinguishable states that never persisted more than a single hourly bar; an empirical audit combining KS/ANOVA distinctness tests with observed dwell times confirmed the corrected guard's selection was the genuine structural elbow in the BIC curve.

3.4.4 Final Fit and Regime Stream Generation

The selected state count is refit once more on the *full* training split (not the BIC-search subsample), with diagonal covariance, 200 EM iterations, tolerance 10^{-4}. Diagonal covariance is used for both the search and the final fit deliberately: an earlier version fit the final model with full covariance while the BIC parameter-count formula above still assumed diagonal covariance, a mismatch that let one state's covariance grow essentially unconstrained during EM, turning it into an "absorbing" catch-all state that any live, short-context HMM query collapsed into regardless of the actual input. Matching covariance structure between the BIC search and the deployed model closes that gap.

The fitted HMM and PCA transform are then applied to the *entire* dataset (train, validation, and test embeddings alike — inference through an already-fit model does not touch the sacred boundary the way fitting would) to produce a regime probability stream: for every asset and timestamp, a hard regime label (Viterbi-style MAP state) and a full probability vector over all n states, both stored for consumption by the agent layer and the RL environment.

Not every regime label is populated for every asset: regime semantics are shaped jointly by all four assets' data during fitting, and some states (e.g. a stress/crisis state) are only ever visited by the forex/commodity series, which have a materially different volatility profile from crypto. Regime label semantics are also not guaranteed stable across refits of the HMM — the mapping from an integer label to an economic interpretation (e.g. "calm bull") is derived post hoc from each state's empirical statistics rather than fixed by construction, so any component that displays a human-readable regime description re-derives it from the current fit's own statistics rather than hardcoding a label-to-description table.

3.5 Multi-Agent Deliberation System

3.5.1 Design Objective

Layer 3 conditions a structured, multi-perspective reasoning process on the regime signal from Layer 2, addressing the interpretability gap identified in Section 2.4.3: rather than a

single LLM call producing an opaque recommendation, five specialized agents each analyze the current market state from a distinct angle, structured as an explicit two-round debate synthesized by a final decision-making agent — the multi-agent debate pattern motivated in Section 2.4.2.

3.5.2 Orchestration and LLM Backend

The debate is orchestrated as a LangGraph state machine. Every agent call goes through a shared base class that wraps a single local LLM, Mistral 7B Instruct served via Ollama with GPU acceleration, called with temperature 0.4, a 600-token generation budget, and the endpoint's native JSON-constrained output mode, under a 300-second timeout with one retry on failure. Because a locally-hosted 7B model does not reliably emit clean JSON even in constrained-output mode, every LLM response passes through a five-stage recovery procedure before being treated as unparseable: (1) direct Pydantic validation of the raw response, (2) extraction from a markdown code fence, (3) regex extraction of the first JSON object anywhere in the text, (4) a greedy first-brace-to-last-brace extraction with trailing-comma repair, and (5) the same greedy extraction with embedded literal newlines collapsed before repair. Only if all five stages fail is a safe, neutral default substituted — and that fallback is always logged, so a systematic parsing failure is visible rather than silently corrupting a downstream decision.

3.5.3 Regime Context

Every agent call is grounded in a shared `RegimeContext` object built once per debate: the regime label and probability vector for the queried asset and timestamp (from Section 3.4), the corresponding market feature window, and any upcoming high-impact macroeconomic events. This ensures all five agents reason about the identical regime signal rather than each re-deriving their own, inconsistent notion of "current conditions."

3.5.4 Agent Roster

3.5.5 Debate Flow

The debate proceeds in three stages. **Round 1**: Macro, Technical, and Sentiment agents each independently produce a thesis (directional bias, confidence, up to four supporting factors, and an assessment of whether their view is aligned, contrary, neutral, or mixed with respect to the current regime); RiskAgent computes its risk limits in parallel, deterministically, with no LLM call. **Contrarian check**: the Contrarian agent evaluates the Round-1 average confidence and, if the 0.60 threshold is met, produces a forced-opposite thesis with historical grounding. **Round 2**: rebuttals are triggered under any of three conditions — the Contrarian is active (which guarantees at least one point of opposition), two Round-1 agents already hold directly opposing biases, or Round-1 shows high average confidence (> 0.75) with no internal disagreement at all (a groupthink check). When none of these hold, Round 2 is skipped entirely and no rebuttals are produced. When active, each participating agent may either maintain or revise its bias and confidence in light of the others' arguments, and the Contrarian itself posts a rebuttal targeting whichever opposing agent holds the highest confidence.

3.5.6 Risk Limit Computation

RiskAgent fits a GARCH(1,1) model on the trailing 90 days of hourly returns for the queried asset (falling back to the rolling sample standard deviation if the GARCH fit fails to converge), producing a one-step-ahead volatility forecast $\hat{\sigma}$. Value-at-risk and Conditional Value-at-Risk at the 95% level are then computed under a normal-tail approximation,

$$\mathrm{VaR}_{95} = \Phi^{-1}(0.05)\,\hat{\sigma}, \qquad \mathrm{CVaR}_{95} = -\frac{\phi(\Phi^{-1}(0.05))}{0.05}\,\hat{\sigma},$$

with Φ the standard normal CDF and ϕ its density. The maximum position size is capped so that a 2%-of-capital daily CVaR budget is never exceeded, $\text{max_position} = \min(1, 0.02/|\mathrm{CVaR}_{95}|)$, and a coarse risk-regime label (low/medium/high/extreme) is assigned from $\hat{\sigma}$ thresholds. This position cap is throttled further, independently of the LLM agents, ahead of scheduled high-impact macroeconomic releases (e.g. FOMC, CPI, NFP): to 50% of its computed value inside a 6-hour window before the event, or 75% inside 24 hours, so that event risk is capped by a hard, auditable rule rather than left entirely to whatever the agents happen to notice in their prompt context.

3.5.7 Fund Manager Synthesis

The FundManager computes a weighted consensus from all Round-1/Round-2 outputs: each agent's final (post-rebuttal, if any) confidence is signed by its directional bias and summed into a bullish score and a bearish score, with the Contrarian's contribution counted at half weight throughout (reflecting its role as a deliberate outlier check rather than an equal vote). Consensus is declared bullish or bearish only if one side exceeds the other by a margin of 0.15; otherwise the debate is recorded as split. A regime-dependent conviction multiplier is then applied — for instance, a bullish consensus is scaled to 40% conviction in a stress/crisis regime and 50% in a bear-trend regime, while a bearish consensus in a calm-bull regime is scaled to 70% — and if the regime unambiguously contradicts the voted direction (bullish consensus during stress/crisis or a bear trend), the decision is overridden to `hold` entirely rather than merely down-weighted. A second, independent override condition fires whenever the Contrarian was active *and* at least one other agent changed its bias after the Round-2 rebuttal — treating a debate that visibly moved an agent's position as evidence the regime override should be flagged even without a hard regime contradiction. The final position size is $\min(0.8 \times \text{conviction},\ \text{max_position_size})$ from RiskAgent, and the FundManager's own LLM call is used only to produce a one-sentence natural-language rationale for the already fully-determined numerical decision — the action, size, and override flags are all computed deterministically before the LLM is consulted, so a parsing failure on the rationale text cannot corrupt the decision itself.

3.6 RL Execution Agent

3.6.1 Design Objective

Layer 4 translates the Layer 3 debate output, together with the regime embedding and current portfolio state, into a concrete position-sizing decision, learned end-to-end with reinforcement learning rather than a fixed sizing rule, so that risk exposure can be modulated by both regime and the debate's own confidence.

3.6.2 MDP Formulation

State (156-dim, clipped to $[-10, 10]$): a 128-dim regime embedding from the frozen Layer-1 encoder (no gradient flows back into it during RL training); a 16-dim agent-consensus vector (per-agent signed confidence, the FundManager's signed conviction, position size, regime-override flag, and dissent count — zeroed when no cached debate exists for the current timestamp, the common case during training, since the debate system is never invoked live during RL training or rollout); an 8-dim portfolio-state block (four canonical per-asset signed position slots, plus normalized running PnL, drawdown, steps-since-last-trade, and the current GARCH volatility forecast); and a 4-dim one-hot asset identifier. The one-hot block was added after a multi-seed audit found that, without it, the shared policy's per-asset behavior was unstable immediately after a curriculum phase introduced a second asset: the only asset-identifying signal available to the network was the portfolio position slot, which reads as zero at the start of every episode and gives the network no reliable way to know which asset it is trading until it has already taken a position.

Action (hybrid): a discrete direction $\in \{\text{long}, \text{short}, \text{hold}\}$ sampled from a Categorical head, and a continuous position fraction $\in (0, 1)$ sampled from a Beta distribution whose shape parameters are constrained to $\alpha, \beta > 1$ via a softplus transform, keeping the distribution unimodal so it cannot collapse mass onto the 0/1 boundary the way an unconstrained Beta could. The Beta distribution was chosen specifically because it is natively bounded on $(0, 1)$, so the position fraction requires no post-hoc clipping and carries no boundary-gradient bias the way a clipped Gaussian head would.

Reward:

$$r_t = \frac{\Delta \text{PnL}_t}{\text{CVaR}_t} - \lambda \cdot \text{drawdown}_t - \text{idle_penalty}_t,$$

where CVaR_t uses the identical Cornish–Fisher-style normal-CVaR formula as RiskAgent (Section 3.5.6), computed from a GARCH(1,1) forecast that is refit on a weekly stride and forward-filled between refits (never refit per environment step, which would make training intractable), and reward is clipped to $[-10, 10]$ to keep gradients well-scaled for the value network. The drawdown penalty is continuous from the first bar of drawdown rather than gated behind a threshold, since a threshold gate leaves small, common drawdowns with zero training signal. The idle penalty is a small constant deduction applied whenever the agent holds a near-zero position, added specifically to break an "always-flat is the safest optimum" local minimum the policy otherwise converges to (Section 3.7); it is a training-time-only signal and is always set to zero during validation and test evaluation, so that abstention is never artificially discouraged when measuring genuine trading performance.

3.6.3 Policy Network

The actor-critic policy shares one MLP trunk ($156 \to 256 \to 128 \to 64$, ReLU activations) feeding three independent linear heads: a 3-way Categorical logits head for direction, a 2-way head producing the Beta distribution's pre-softplus (α, β) parameters, and a scalar value head. This required overriding Stable-Baselines3's default policy-construction and action-distribution machinery directly, since SB3's built-in PPO implementation supports only a single, homogeneous action distribution and has no native support for a hybrid discrete/continuous action space.

3.6.4 Curriculum Training

Training follows a curriculum, motivated by the general practice (Section 2.5.3) of mastering a single, liquid asset before extending to less liquid or structurally different ones. Four phases are defined — BTC only; BTC+ETH; +EUR/USD; +XAU/USD — each with its own validation-Sharpe gate that must be met before the shared policy is allowed to advance, and the environment's active asset list is mutated in place between phases rather than restarting training, so that policy weights (and therefore any cross-asset transfer) persist across the whole curriculum. In practice, the two forex/commodity phases were found to actively erode the BTC/ETH policy learned in the first two phases: because histdata.com's underlying intraday bars carry comparatively little genuine hourly signal relative to the crypto pair, "hold" is close to the reward-minimizing-variance action for those assets, and continuing training into Phases 3–4 pulls the shared policy toward an always-flat behavior that also degrades its BTC/ETH performance. Production training therefore caps the curriculum at Phase 2 (BTC+ETH) by default; Phases 3–4 remain fully implemented and are used only for the explicit ablation and diagnostic runs described in Section 3.7, not for the headline checkpoint.

Eight parallel environments, each starting from an independently randomized start point within the active split, are used specifically to decorrelate the rollout buffer: a single environment produces one long autocorrelated trajectory per buffer, which was found to destabilize training, whereas 8 independent trajectories give PPO a buffer closer to the i.i.d. assumption its advantage estimates rely on. The discount factor was lowered from an initial 0.99 (an effective $\sim$100-step horizon) to 0.95, and the entropy coefficient is annealed from a comparatively high 0.05 down to 0.005, both changes made after diagnosing that a lower-entropy, longer-horizon initial configuration collapsed too quickly to an always-hold policy before it could discover profitable trading behavior.

3.6.5 Evaluation Determinism

Because the validation window contains relatively few meaningful trades, small sources of non-determinism were found to be sufficient to flip the sign of the measured Sharpe ratio between otherwise-identical evaluation runs. Two concrete sources were identified and fixed: the frozen encoder's forward pass ran on GPU (unseeded cuDNN kernels) once per environment step during evaluation, and neither PyTorch nor NumPy's global RNG was seeded before an evaluation pass. All evaluation-only environments and model loads are now forced onto CPU (with the shared encoder module deep-copied before being moved, since moving it in place would otherwise silently drag a live training environment's encoder onto CPU too), and both RNGs are seeded immediately before every evaluation pass. Repeated evaluation runs on an identical checkpoint now return bit-identical metrics.

3.7 Evaluation Methodology

3.7.1 Metrics

Trading performance is summarized by the annualized Sharpe ratio (hourly returns annualized by $\sqrt{24 \times 365}$), maximum drawdown, and Calmar ratio, alongside execution-level diagnostics: because the continuous Beta position head produces a nonzero-sized change on almost every step, a raw trade count is not a meaningful measure of trading frequency,

so a trade is only counted as *meaningful* once the position change exceeds 0.5% of capital, and mean turnover is reported separately as the actual measure of execution activity.

3.7.2 Ablation Design

The central empirical question of this work — whether regime-awareness measurably improves risk-adjusted trading performance — is answered through a seven-variant ablation isolating the contribution of the regime embedding, the multi-agent consensus, and the PPO execution layer independently:

Each variant's result records which of these methods actually produced it, so that, for instance, a reader comparing `ppo_no_regime` against `marta_full` is comparing "momentum execution without regime context" against "the trained policy with regime context," not silently comparing against a degenerate always-flat run mislabeled as a fair baseline.

3.7.3 Statistical Testing

Statistical significance of the Sharpe-ratio delta between `marta_full` and each baseline is assessed with a block bootstrap (1000 resamples, fixed seed) rather than an i.i.d. bootstrap over individual hourly returns: returns are resampled in non-overlapping 168-hour (one-week) blocks, preserving the autocorrelation structure of the underlying return series, which an i.i.d. resampling scheme would otherwise understate. The reported delta is the bootstrap mean of the per-resample Sharpe difference, with the 2.5th/97.5th percentiles reported as a 95% confidence interval; returns are trimmed to whole blocks before resampling.

3.7.4 In-Sample Regime Behavior Diagnostic

A separate, explicitly diagnostic-only probe examines how the trained agent sizes positions across known crisis and bear episodes that fall inside the *training* split — the March 2020 crash and the 2018 crypto bear market — reporting average position size per decision-time regime and per-regime-transition position changes. Because both windows are in-sample, this analysis is never combined with, or presented alongside, the out-of-sample validation/test metrics above; it answers a qualitative question (does the policy's behavior differ meaningfully across regimes it was trained on) rather than a performance claim.

3.8 Dashboard and Live System

3.8.1 Design Objective

Layer 5 exposes regime detection, agent reasoning, and execution decisions from Layers 1–4 in a live, auditable interface, directly supporting the interpretability property identified as largely absent from prior work in Section 1.3.1.

3.8.2 Architecture

The backend is a FastAPI application; the frontend is a Next.js single-page application. Scheduling is handled by APScheduler, running an hourly cron-triggered signal tick rather than a message-queue-based task system, since the system's decision cadence (one debate

and one execution decision per asset per hour, matching the pipeline's native hourly resolution) does not require queue-based task distribution. Real-time price, regime, and debate state are pushed to the frontend over WebSocket connections; trade execution runs against Alpaca's paper-trading API, with Telegram used for out-of-band alerting and a chat interface for ad hoc natural-language queries against the live system state.

3.8.3 Live Regime Lookup

Regime lookup exists in two distinct forms that are deliberately not interchangeable. A static lookup resolves the nearest row in the pre-computed regime probability stream (Section 3.4.4) for a given historical timestamp, which is correct and cheap for genuinely past bars but is a frozen snapshot that stops advancing the moment the offline regime pipeline last ran. A live lookup instead recomputes PCA and HMM inference fresh from a rolling 168-hour embedding sequence ending at the query time; a single-frame HMM query was tried first and found to be dominated by the model's stationary prior rather than the actual current observation, which is why a full sequence, not a single point, is required for a live query to actually move. Every component that reasons about "now" (the live price feed's regime tag, the chat assistant, the hourly signal tick) uses the live lookup; only genuinely historical queries use the static one.

3.8.4 Live Feature Bridge

Because EUR/USD and XAU/USD's training-time data is capped at 2025-12-31 (Section 3.2.1), a separate live feature bridge pulls real hourly OHLC bars from a market-data API and reproduces the exact same feature-engineering and normalization functions used at training time (Section 3.2.2), rather than a parallel reimplementation, appending them onto an append-only raw-bar accumulator that is itself never passed back through feature computation. This last detail matters operationally: an earlier version fed its own already-warmup-trimmed output back in as the next cycle's context, losing tens of rows to indicator warm-up against a single new bar per hourly cycle, which erodes the usable history to nothing within days; keeping a separate, untouched accumulator avoids that failure mode. BTC and ETH's own training-time data reaches materially further forward already and do not yet have an equivalent bridge, since the resulting staleness for those two assets is measured in weeks rather than months.

3.8.5 Auditability

Every quantity displayed alongside a trading decision — win rate, best/worst trade, position history — is sourced from the same evaluation run that produced the headline Sharpe ratio shown next to it, rather than a separately generated snapshot that could silently drift out of sync with the number it is displayed beside; this was a deliberate design correction after an earlier version combined numbers from two different data sources on the same dashboard card.

Table 3.4: The six agents of Layer 3 (five LLM-based deliberation agents plus one deterministic risk-limit agent)

Agent	**Role**
MacroAgent	Cross-asset correlation matrix and contagion flag computed over a 168-candle (7-day) lookback window; LLM interprets the resulting statistics into a directional thesis.
TechnicalAgent	Momentum and volatility indicators plus the encoder's own VSN attention weights (Section 3.3.2) over a 60-candle window — the only agent that consumes the encoder's internal feature-importance signal directly.
SentimentAgent	FinBERT sentiment scores over news headlines within a 24-hour lookback window, aggregated with exponential recency decay (half-life 6 h); a 6-hour-vs-24-hour momentum comparison is also computed. FinBERT produces the quantitative scores; the LLM call interprets them into a narrative thesis, it does not re-score sentiment itself.
RiskAgent	The only non-LLM agent: fits a GARCH(1,1) model on the lookback window's hourly returns, converts the resulting one-step-ahead volatility forecast into 95% VaR/CVaR under a Gaussian-tail assumption, and derives a hard maximum position size and leverage cap (Section 3.5.6).
ContrarianAgent	Devil's-advocate agent; activates only when the Round-1 average confidence across the other agents reaches 0.60. When active, its `directional_bias` is *forced* to the opposite of the emerging consensus regardless of what the LLM itself argues, and it retrieves historical analogues — past occurrences of the same regime label with probability > 0.80 — reporting the empirical loss rate and average 24-hour forward return across up to 100 such cases, to ground its counter-argument in data rather than rhetoric alone.
FundManagerAgent	Synthesizes the full debate (Round-1 theses, Round-2 rebuttals, risk limits) into a single final decision (Section 3.5.7); the only agent whose output is actually executable.

Table 3.5: PPO training configuration

Parameter	**Value**	**Parameter**	**Value**
Learning rate	3×10^{-4}	Parallel environments	8
Rollout length (per env)	512	Rollout buffer size	4096
Minibatch size	64	Epochs per update	10
Discount γ	0.95	GAE λ	0.95
PPO clip range	0.2	Value-loss coefficient	0.5
Entropy coefficient	$0.05 \rightarrow 0.005$ (decayed over 100k steps)	Max gradient norm	0.5
Drawdown penalty λ	0.01	Max steps/episode	2000

Table 3.6: Ablation variants

Variant	**Regime / Agents / PPO**	**Method**
`buy_hold`	— / — / —	Static long-only baseline.
`rsi_strategy`	— / — / —	Rule-based RSI(14) mean-reversion (overbought 70 / oversold 30).
`ma_crossover`	— / — / —	Rule-based moving-average(12,26) crossover.
`ppo_no_regime`	— / — / PPO	Zeroing the 128-dim regime slot pushes the trained policy out-of-distribution and it collapses to constant-hold, which would trivially read as "identical to hold" rather than isolating regime dependence; this variant instead runs a 3-bar unanimous-momentum rule at fixed 0.3 position size, representing "RL-like execution without regime awareness."
`agents_no_regime`	— / Agents / —	Rule-based on the cached multi-agent debate consensus, using nearest-debate interpolation (snapping each step to the closest cached debate within 7 days) since the debate cache is sparse relative to the evaluation window; FundManager position size is capped at 5% to keep this variant's sizing comparable in scale to PPO's own risk-capped output.
`ppo_agents_no_regime`	— / Agents / PPO	As `ppo_no_regime`, with position size additionally scaled by the nearest cached debate's confidence.
`marta_full`	Regime / Agents / PPO	The complete system.

Bibliography

[1] James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

[2] Market regime detection using hidden markov models. QuestDB Glossary. Accessed 2026.

[3] Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*, 2020.

[4] Hmm-based market regime detection with rl for portfolio management. *International Conference on Data Security and Privacy Protection (DataSec)*, 2025.

[5] Wentao Zhang et al. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. 2024.

[6] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.

[7] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[8] Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454